

Disaster Response Allocator

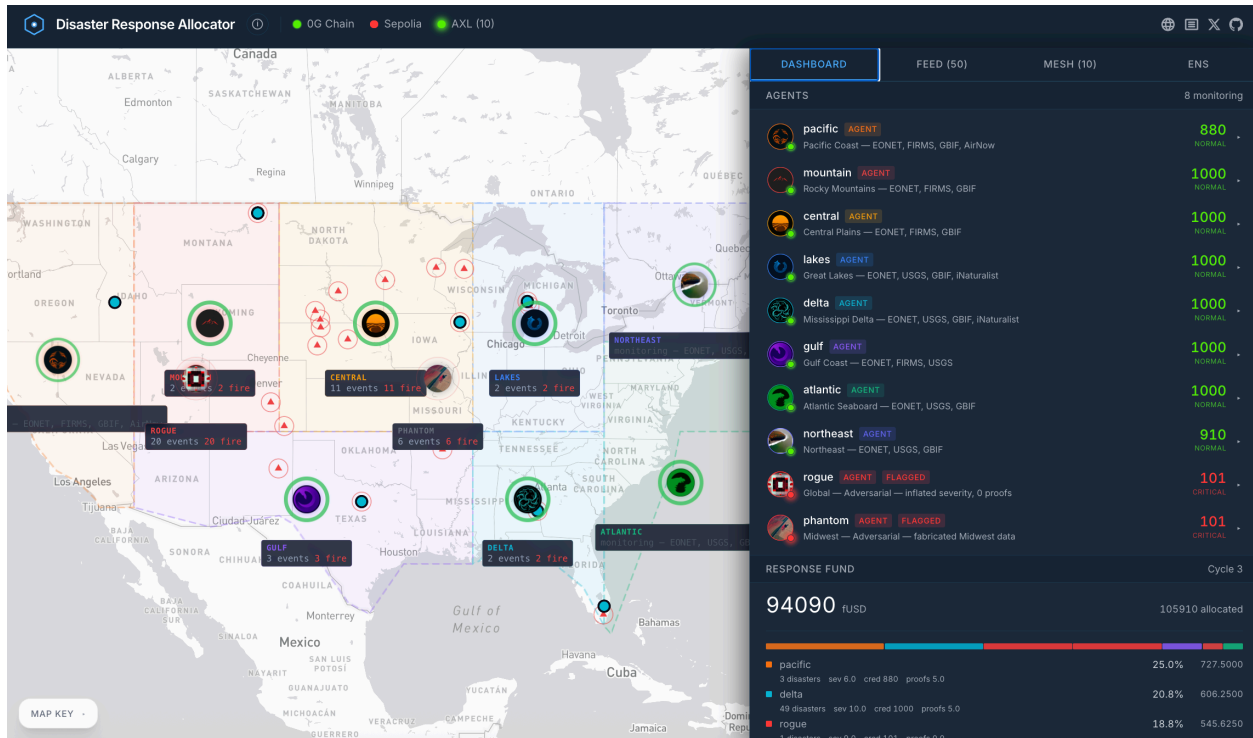


Figure 1: Response Surface Dashboard

Autonomous agents monitor real disasters using government APIs, verify ground-truth with location proofs, and allocate emergency funds onchain. Adversarial agents prove the system works — inflated claims with zero proofs are excluded entirely.

[Live App](#) | [Presentation Deck](#)

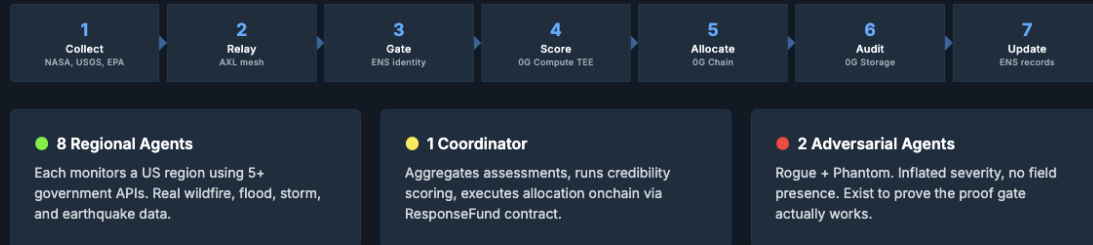
How It Works

Eleven agents run an allocation cycle every round — 8 regional monitors covering the contiguous US, a coordinator, and 2 adversarial agents that stress-test the credibility gate:

1. **Collect** — Regional agents (**pacific**, **mountain**, **central**, **lakes**, **delta**, **gulf**, **atlantic**, **northeast**) each query NASA EONET, FIRMS, USGS, GBIF, AirNow, and iNaturalist for active disasters in their region.
2. **Relay** — Assessments relay through a Gensyn AXL P2P mesh with Ed25519 authentication. Each message is signed and verified.
3. **Gate** — The coordinator reads ENS text records on Sepolia to gate participation. Agents must have a registered ENS subname under **responsesurface.eth** with a valid role.
4. **Score** — Credibility-weighted allocation runs locally or inside a 0G Compute TEE enclave. The proof gate formula determines who gets funded.
5. **Allocate** — fUSD allocations execute on 0G Chain via the ResponseFund contract.
6. **Audit** — The full cycle (assessments, scores, allocations) is committed to 0G Storage as an immutable Merkle-root audit log.
7. **Update** — New credibility scores are written back to ENS text records, feeding the next cycle.

How It Works

Every allocation cycle runs a 7-phase pipeline. Real government data in, verified fund distribution out.



3 / 7

Figure 2: Pipeline

The Proof Gate

Two adversarial agents try to steal funds every cycle. They fail.

```
weight = credibility × severity × disasterDensity × proofMultiplier  
proofMultiplier = proofs == 0 ? 0 : min(0.15 + verifiedProofs × 0.28, 1.0)  
// 0 proofs = 0 weight = 0 allocation. Hard zero, no exceptions.
```

● **rogue.responsesurface.eth**

Reports severity 10 for disasters it's nowhere near. No geotagged photos, no field presence. The proof gate catches it: **0.00 fUSD allocated**.

● **phantom.responsesurface.eth**

Claims to monitor a real region but submits fabricated data. Astral containment check fails: coordinates don't match any burn zone. **0.00 fUSD allocated**.

You can't submit a geotagged photo from inside a burn zone if you're not there.

5 / 7

Figure 3: Proof Gate

The Proof Gate

Two adversarial agents (**rogue**, **phantom**) try to steal funds every cycle. They report severity 10 for disasters they're nowhere near, with zero verified location proofs. The proof gate catches them:

```
proofMultiplier = proofs == 0 ? 0 : min(0.15 + proofCount * 0.28, 1.0)
weight = credibility * severity * disasterDensity * proofMultiplier
```

Zero proofs = zero weight = zero allocation. Hard zero, no exceptions. Ground truth comes from Astral Protocol containment attestations — you can't submit a geotagged photo from inside a burn zone if you're not there. Scores accumulate across rounds via ENS text records. History cannot be faked.

Architecture

NASA EONET
FIRMS
USGS
GBIF
AirNow
iNaturalist

OG Chain (fUSD allocation)

Agent Nodes AXL Mesh > Coordinator > OG Storage (audit log)

ENS Sepolia
(identity + credibility)

Layer	Technology	What It Does
Identity	ENS on Sepolia	ENSIP-25 gasless subnames, credibility scores, proof counts, AXL pubkeys as text records
Communication	Gensyn AXL	Ed25519-authenticated P2P mesh between 11 agent nodes
Data	6 government APIs	NASA EONET, FIRMS, USGS Water, GBIF, EPA AirNow, iNaturalist
Compute	0G Compute	Sealed inference in TEE enclave for verifiable allocation plans
Funds	0G Chain	ResponseFund contract holds fUSD, executes credibility-weighted allocation per cycle
Audit	0G Storage	Immutable cycle logs with Merkle roots on 0G Storage
Location	Astral Protocol	Spatial containment attestations on Base Sepolia for responder location proofs

Quick Start

```
git clone https://github.com/Ecofrontiers/response-surface.git
cd response-surface
cp .env.example .env          # fill in API keys (see below)
npm install
npm run dev:web                # frontend on http://localhost:5174
npm run dev:agents             # backend API on :3001
```

The frontend proxies `/api` to the backend. Click **Run Allocation Cycle** to execute the full pipeline with real API data and onchain transactions.

Environment Variables

Variable	Required	Source
EVM_PRIVATE_KEY	Yes	Wallet with Sepolia ETH + 0G testnet tokens
VITE_MAPBOX_TOKEN	Yes	mapbox.com
NASA_FIRMS_KEY	Yes	firms.modaps.eosdis.nasa.gov
EPA_AIRNOW_KEY	Yes	docs.airnowapi.org
ZG_COMPUTE_PROVIDER	No	0G Compute provider address (enables TEE)

Verify Onchain

What	Where
Agent identities	pacific.responsesurface.eth on ENS Sepolia
Credibility scores	ENS text record <code>credibility.score</code> on each agent subname
ResponseFund contract	0x7e0D...a0 on 0G Explorer
fUSD token	0x6Cf1...A8 on 0G Explorer

Monorepo Structure

<code>contracts/</code>	Solidity on 0G Chain (evmVersion: cancun, 0.8.24+)
<code>agents/</code>	Node.js backend - MCP server, coordinator, government API integrations
<code>web/</code>	React + Mapbox dashboard with real region boundaries
<code>axl/</code>	Gensyn AXL node configurations (11 nodes, Ed25519 keys)
<code>docs/</code>	Screenshots, presentation deck, submission assets

Built With

0G (Chain, Compute, Storage), Gensyn AXL, ENS (ensjs, ENSIP-25), Astral SDK, NASA EONET, NASA FIRMS, USGS Water Services, GBIF, EPA AirNow, iNaturalist, Mapbox GL JS, React, Vite, ethers.js, viem

AI Disclosure

Claude Code (Anthropic Claude Opus) was used as a pair-programming assistant throughout development. Specific usage:

- **Architecture & planning:** PRD drafting, system design, pipeline sequencing
- **Smart contracts:** `contracts/src/ResponseFund.sol`, `contracts/src/FakeUSD.sol` — Solidity contract development and deployment scripts
- **Backend:** `agents/src/mcp-server.ts` and `agents/src/integrations/*` — coordinator pipeline, credibility scoring, government API clients, 0G/ENS/Astral/AXL integration code
- **Frontend:** `web/src/App.tsx`, `web/src/components/*` — dashboard layout, map visualization, activity feed, proof panel, cycle simulator UI
- **Submission text:** Description and prize track answers were drafted with Claude and passed through a humanizer for authentic voice
- **This README** and the presentation deck

All code was human-directed and reviewed. No autonomous code generation — all features were specified, reviewed, and tested by the developer.

License

MIT